Rendu TP1 : Java RMI

Meryam RHADI
Lamyae KHAIROUN

7 novembre 2021

Master 1
génie logiciel

UE Architectures logicielles Distribuées

Responsable
Abdelhak-Djamel Seriai
Bachar Rima

Sommaire

Titre 1

Sommaire 2

1 Conception de la solution 3
1.1 Diagramme de classe : 3

2 Description des packages : 4
2.1 package Common 4
2.2 package Server 4
2.3 package Client 5

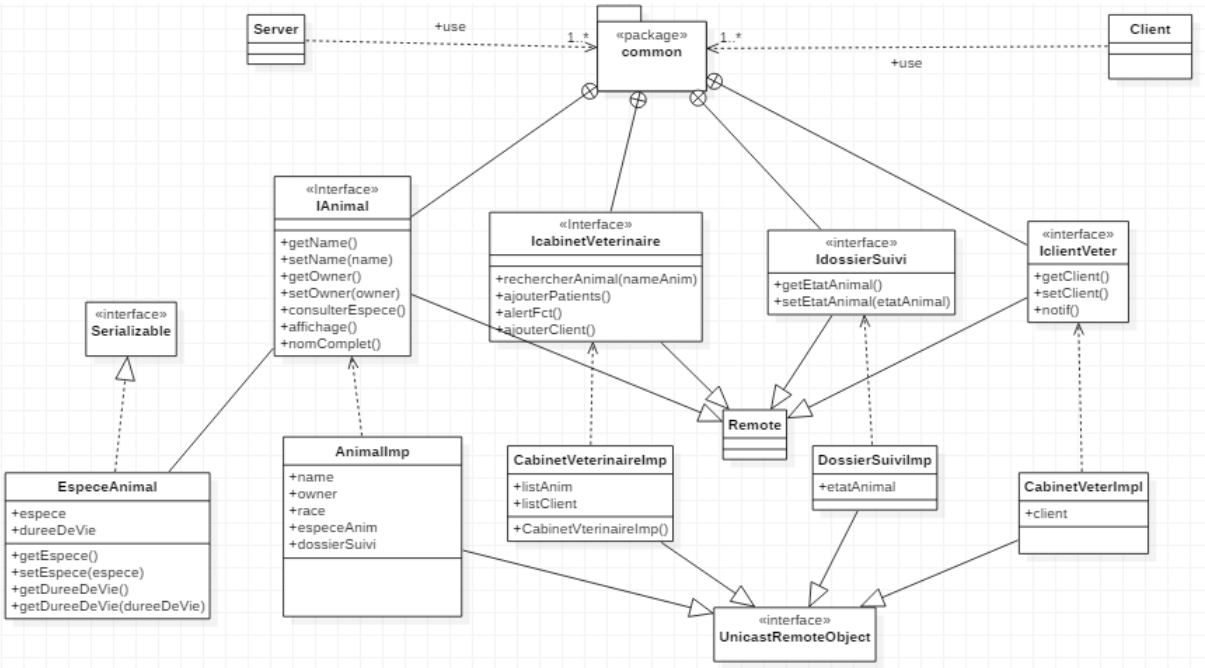
3 Capture du résultats : 6
3.1 Cote serveur : 6
3.2 Cote client : 6

4 Read Me 6

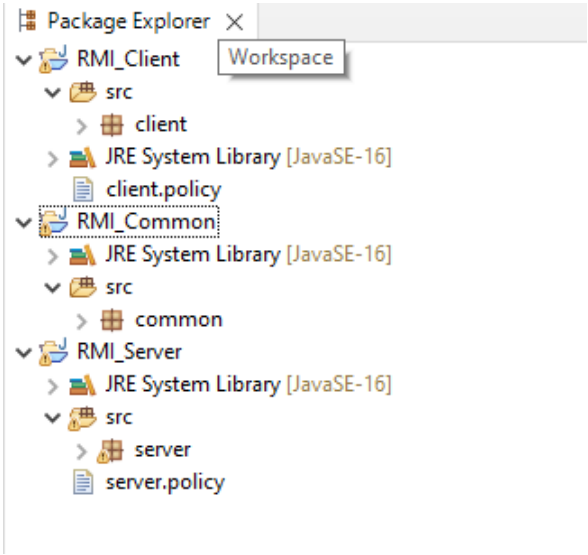
1 Conception de la solution

Pour manipuler ce TP on a crée 3 projets différents (Client, Server et Common).

1.1 Diagramme de classe :



2 Description des packages :



2.1 package Common

Ce projet contient les entités communes entre le Client et le Serveur, comme les interfaces des objets distants, dans notre cas les interfaces représentants les objets distants sont :
- **IAnimal :** qui contient les attributs et les méthodes à appliquer sur un animal.
- **ICabinetVeterinaire :** contient la liste des animaux qui se trouve dans une cabinet vétérinaire, ainsi que les méthodes à appliquer sur cette liste.
- **IclientVeter :** contient la liste des clients de la cabinet vétérinaire et les méthodes en relation avec celle-ci.
- **IdossierSuivi :** contient les attributs qui donnent des informations sur l'état d'un animal est alors son dossier de suivi qui est représenter pour le moment par une chaîne de caractères.

2.2 package Server

Ce projet contient la classe Server et les classes qui devraient être reconnues par le serveur via le codebase. il contient les implémentation des interfaces qui sont dans common, ces dernier héritent tous le comportements de UnicastRemoteObject :
AnimalImp : Qui implémente IAnimal, on a ajouté les attributs (name, owner,race) de type String et especeAnimal de type EspeceAnimal et dossierSuivi de type DossierSuivi. On a ajouté les méthodes usuels (le

constructeurs et les getters/setters), la méthode consulterEspece qui retourne le nom de l'espèce et sa durée de vie on utilisant les getters de la classe EspeceAnimal, la méthode nomComplet qui retourne le nom et le maitre d'un animal, et finalement la méthode d'affichage qui affiche tous les informations d'un animal.
DossierSuiviImp : implémente l'interface DossierSuivi , il a un seul attribut etatAnimal de type String qui sert à décrire l'état d'un animal, de plus on a ajouté les méthodes get/set.
EspeceAnimal : Il implémente serializable pour passer un objet d'une machine à l'autre, et qui n'est pas manipulé directement par le client.
CabinetVeterinaireImpl : Cette classe implémente l'interface ICabinet-Veterinaire, on a ajouté comme attributs une liste de AnimalImpl qui sert à stocker les animaux qui suivent leurs traitement dans le cabinet, ainsi qu'une liste de IclientVeter, qui stocke tous les clients du cabinet. Cette classe implémente les méthodes suivantes :
- rechercherAnimal : qui recherche un animal par son nom dans la liste des animaux et le retourne s'il existe, sinon elle retourne null.
- ajouterPatient : qui sert à ajouter un nouveau patient dans la liste des animaux et le retourne.
- alerteFct : cette fonction notifie les client d'une même cabinet veterinaire lorsqu'on franchit la hausse ou la baisse les seuils 100, 500, et 1000.
- ajouterClient : cette méthodes sert à ajouter un nouveau client dans la liste des clients.

Server : Cette classe à pour rôle de crée une instance de l'implémentation de l'objet distant, exporte l'objet distant, attend les requêtes via le squelette, et il enregistre les objet distant dans le registre d'objets distants RMI.

2.3 package Client

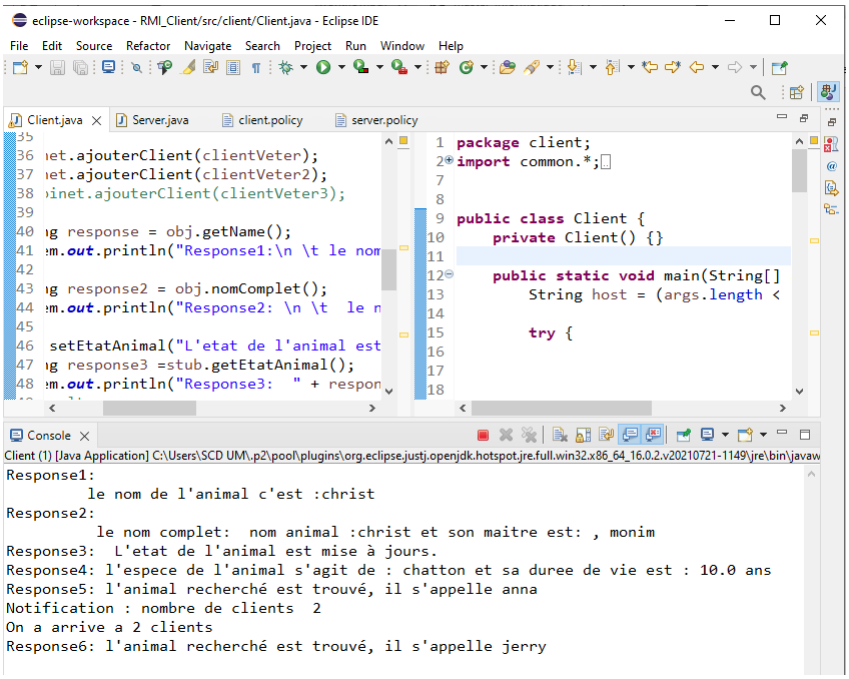
Cette classe à pour rôle d'obtenir un stub pour le registre sur l'hôte du serveur en appelant la méthode statique LocateRegistry.getRegistry, recherche le stub de l'objet distant par nom dans le registre , et finalement il appele les méthodes dont il a besoin sur l'objet distant à l'aide du stub. On a ajouté aussi l'implémentation de l'interface IclientVeter dans le projet du client, cette classe a un seule attribut client de type String qui represente le nom du client,en plus on a ajouté les méthodes get/set, et une méthode notif qui affiche une phrase pour notifier le client du nombre totale des clients qui existe.

3 Capture du résultats :

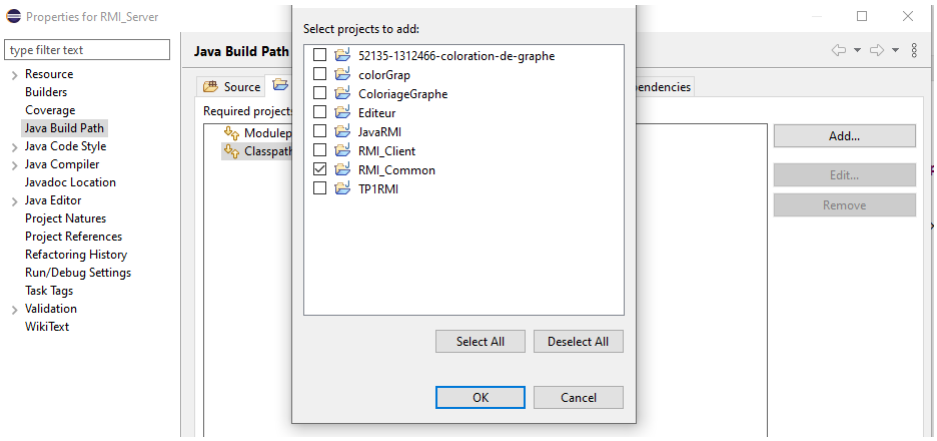
3.1 Cote serveur :

```
Server ready
Objet trouve : anna
Debut d'alerte
Objet: jerry est non trouve
Objet: jerry est non trouve
Objet: jerry est non trouve
Objet: jerry est non trouve
Objet: jerry est non trouve
Objet trouve : jerry
```

3.2 Cote client :



4 Read Me

Pour pouvoir exécuter le projet, il faut télé-charger et extraire l'archive de projet.
Sur eclipse il faut ajouter le code base du projet Common dans les deux autre projet (Client/Server), pour ce faire :
clique droit sur le projet Client/Server, –> Build Path –> Configure Build

Path –> Class Path –> Add –> Choisir le projet Common –> cliquer sur Apply.